# EE5203 L05 Study Sheet

*GPIO and Hardware Abstraction - Basri Erdogan*

**Essential question:** `HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_SET)` and `GPIOA->BSRR = (1U << 5)` produce the same hardware result — what exactly does the HAL layer add, and how do you verify it wrote the bits you expect?

## What you should be able to do

- Read a `GPIO_InitTypeDef` configuration and predict the register fields it produces.
- Map `GPIO_MODE_OUTPUT_PP`, `GPIO_NOPULL`, `GPIO_SPEED_FREQ_LOW` to `MODER`/`OTYPER`/`PUPDR`/`OSPEEDR` bits.
- Use `HAL_GPIO_WritePin`, `TogglePin`, `ReadPin` correctly.
- Explain floating inputs, pull-up/pull-down, and why B1 is active-low.
- Detect a button **edge** (with debounce) rather than a level.
- Verify any HAL configuration in the SFR view.

## The abstraction stack

```
your code -> STM32 HAL -> CMSIS register -> hardware pin
led_on()     HAL_GPIO_WritePin()   GPIOA->BSRR   PA5 / LD2
```

HAL changes how you *write* the command, not which hardware receives it. Neither interface removes the need to verify the result.

## Configuration: `GPIO_InitTypeDef`

```
GPIO_InitTypeDef config = {0};      /* zero every member first     */

config.Pin   = GPIO_PIN_5;
config.Mode  = GPIO_MODE_OUTPUT_PP;
config.Pull  = GPIO_NOPULL;
config.Speed = GPIO_SPEED_FREQ_LOW;

HAL_GPIO_Init(GPIOA, &config);      /* address: the function READS
                                       your members through it     */
```

| HAL name | Register effect (PA5) |
|---|---|
| `GPIO_PIN_5` | selects pin 5 (a mask, like `1U << 5`) |
| `GPIO_MODE_OUTPUT_PP` | `MODER[11:10] = 01`, `OTYPER[5] = 0` |
| `GPIO_NOPULL` | `PUPDR[11:10] = 00` |
| `GPIO_SPEED_FREQ_LOW` | `OSPEEDR[11:10] = 00` |

For PC13 the same fields live at bits `27:26` (`13 x 2 = 26`).

**Speed is not blink rate:** `Speed` sets the electrical transition strength; `HAL_Delay` sets the visible timing.

## Runtime: the three functions

| Function | Job | Returns |
|---|---|---|
| `HAL_GPIO_WritePin(port, pin, state)` | force high/low | — |
| `HAL_GPIO_TogglePin(port, pin)` | reverse output | — |
| `HAL_GPIO_ReadPin(port, pin)` | sample input | `GPIO_PIN_SET` / `GPIO_PIN_RESET` |

Configuration happens once (`MX_GPIO_Init()`, called by `main()` before the loop); these run inside the loop.

## Inputs: floating, pulled, and B1

- **Floating:** nothing defines the level — noise decides what you read.

- **Pull-up** (`PUPDR = 01`): released reads `1`. **Pull-down** (`10`): released reads `0`. **None** (`00`): the external circuit must decide.
- **B1 on PC13** has an **external** pull-up (R30) and grounds the pin when pressed → configure `GPIO_NOPULL`, and the logic is **active-low**:

| B1 | PC13 | `HAL_GPIO_ReadPin` |
|---|---|---|
| released | high | `GPIO_PIN_SET` |
| pressed | low | `GPIO_PIN_RESET` |

## Edge detection, not level

```c
static uint32_t prev = 1U;                          /* released */
uint32_t cur = (uint32_t)HAL_GPIO_ReadPin(GPIOC, GPIO_PIN_13);

if (prev == 1U && cur == 0U) {   /* falling edge = new press */
    ++press_count;
    HAL_Delay(10);               /* debounce */
}
prev = cur;
```

Testing `cur == 0U` alone counts continuously while the button is held; the `prev`/`cur` pair fires once per press. The short delay rides out contact bounce.

## Ownership rule

CubeMX owns generated init; USER CODE regions own your calls; the SFR view provides the evidence; direct register writes only when the exercise assigns them. Never let CubeMX and hand-written register code fight over one pin.

## Common mistakes

- `MX_GPIO_Init()` never runs (or is called twice) before the loop.
- Port/pin mismatch: `GPIOA` with `GPIO_PIN_13`, or `GPIOC` with `PIN_5`.
- Wrong pull for the circuit — internal pull-up added on PC13 is harmless but wrong-by-specification; missing pull on a truly floating input is a bug.
- Reversed logic: treating `GPIO_PIN_SET` as pressed.
- Edits outside USER CODE regions vanish on regeneration.
- Confusing `Speed` with blink timing.

## Mastery self-check

- ☐ I can write the four `GPIO_InitTypeDef` members for PA5 output from memory.
- ☐ I can give the `MODER`/`PUPDR` field values HAL writes for PC13 input, no pull.
- ☐ I can explain why `HAL_GPIO_Init` takes `&config`.
- ☐ I can state what B1 pressed reads and why.
- ☐ I can write the edge-detection pair and explain the debounce delay.
- ☐ I can list the three SFR checks that verify the lab's configuration.
- ☐ I can argue one strength and one cost of each interface.

## Five review questions

1. `HAL_GPIO_Init(GPIOC, &config)` ran with `Mode = GPIO_MODE_INPUT`, `Pull = GPIO_NOPULL`, `Pin = GPIO_PIN_13`. Which register fields changed, and to what values?
2. Why does passing `config` (without `&`) not compile — and if it did, what Week-4 concept explains why it still could not work?
3. A student configures PC13 with `GPIO_PULLUP`. Does the button still work, and what physically changes on the pin?
4. The press counter increases by 3 for one press. Name the two distinct causes and the fix for each.
5. In `stm32f4xx_hal_gpio.c`, `HAL_GPIO_WritePin` ends in a single `BSRR` write. Why does that make plain `=` safe there — and what does this tell you about what HAL is *made of*?

See the L05 worksheet for extended practice. Worked solutions are released after the practice window.